Table of Contents

SentinelX — Secure API Gateway & Security Operations Dashboard 1
Table of Contents 1
1. The 30-second pitch 2
2. What problem does it solve? 2
3. Tech stack (and why each piece) 3
4. Architecture — how it all connects 4
5. The security concepts (interview gold) 5
6. The request pipeline, step by step 8
7. Backend — every module explained 8
8. Backend — every API endpoint 9
9. Frontend — every page explained 11
10. Database tables 12
11. How to run it from scratch 13
12. How to demo it (live walkthrough script) 14
13. Interview Q&A 15
14. Troubleshooting 15
15. Project structure 16
16. Roadmap / planned refinements 16

SentinelX — Secure API Gateway & Security Operations Dashboard

> **Read this one file and you can explain the entire project in any interview.** It covers what the project is, every concept it demonstrates, how every piece works, how it all connects, and exactly how to run it from scratch.

Table of Contents

1. The 30-second pitch
2. What problem does it solve?
3. Tech stack (and why each piece)
4. Architecture — how it all connects

5. The security concepts (interview gold)
6. The request pipeline, step by step
7. Backend — every module explained
8. Backend — every API endpoint
9. Frontend — every page explained
10. Database tables
11. How to run it from scratch
12. How to demo it (live walkthrough script)
13. Interview Q&A
14. Troubleshooting
15. Project structure
16. Roadmap / planned refinements

---

1. The 30-second pitch

> "SentinelX is a **secure API gateway** — a security layer that sits between clients and a backend. Before any request reaches the backend, the gateway **authenticates** it with JWT, **authorizes** it with role-based access control, checks it for **replay attacks** and **tampering**, applies **rate limiting**, calculates a **risk score**, **logs** everything, and returns a **digitally signed** response. On top of the backend there's a premium React **security operations dashboard** to visualize all of this — with charts, an attack simulator, an encryption visualizer, and a live audit log. It demonstrates JWT auth, RBAC, AES-256 + RSA-2048 encryption, SHA-256 integrity, and digital signatures — the core building blocks of real API security."

In one line: *A mini API-security platform that authenticates, encrypts, verifies, scores, and logs every request — with a beautiful dashboard to watch it happen.*

---

2. What problem does it solve?

APIs are the front door to every modern app, and that door gets attacked: stolen tokens, replayed requests, tampered payloads, brute-force logins, abuse through flooding. Normally these protections are scattered and invisible.

A gateway centralizes them into one checkpoint. Think of an **airport security line**:

Airport	SentinelX gateway
Show your passport	**JWT verification** — are you a real, logged-in user?
Boarding pass = seat 4B	**Role check (RBAC)** — are you allowed *here*?

Airport	SentinelX gateway
Ticket already scanned?	**Replay detection** — is this a duplicated request?
Crowd rushing one gate	**Rate limiting** — too many requests too fast?
Baggage X-ray	**SHA-256 integrity** — was the payload tampered with?
Sealed diplomatic pouch	**AES/RSA encryption** — keep the contents secret
Officer's judgment call	**Risk score** — allow / flag / reject
CCTV records everything	**Audit logging**

Every feature in this project is one of those gates, made **visible** through the dashboard.

---

3. Tech stack (and why each piece)

Backend

Package	Why it's here
Flask	Lightweight Python web framework — turns Python functions into API endpoints.
Flask-SQLAlchemy	ORM: lets us define database tables as Python classes and swap SQLite ↔ MySQL with a one-line change.
Flask-JWT-Extended	Creates and verifies JWT tokens; provides the `@jwt_required` decorator.
bcrypt	Securely hashes passwords (slow-by-design so brute force is impractical).
PyCryptodome	The crypto engine: AES-256, RSA-2048, SHA-256, digital signatures.
PyMySQL	MySQL driver (used when we deploy; SQLite is used locally).
python-dotenv	Loads secrets from a `.env` file so they never live in code.
flask-cors	Lets the React frontend (a different origin) call the API.

Frontend

Package	Why it's here

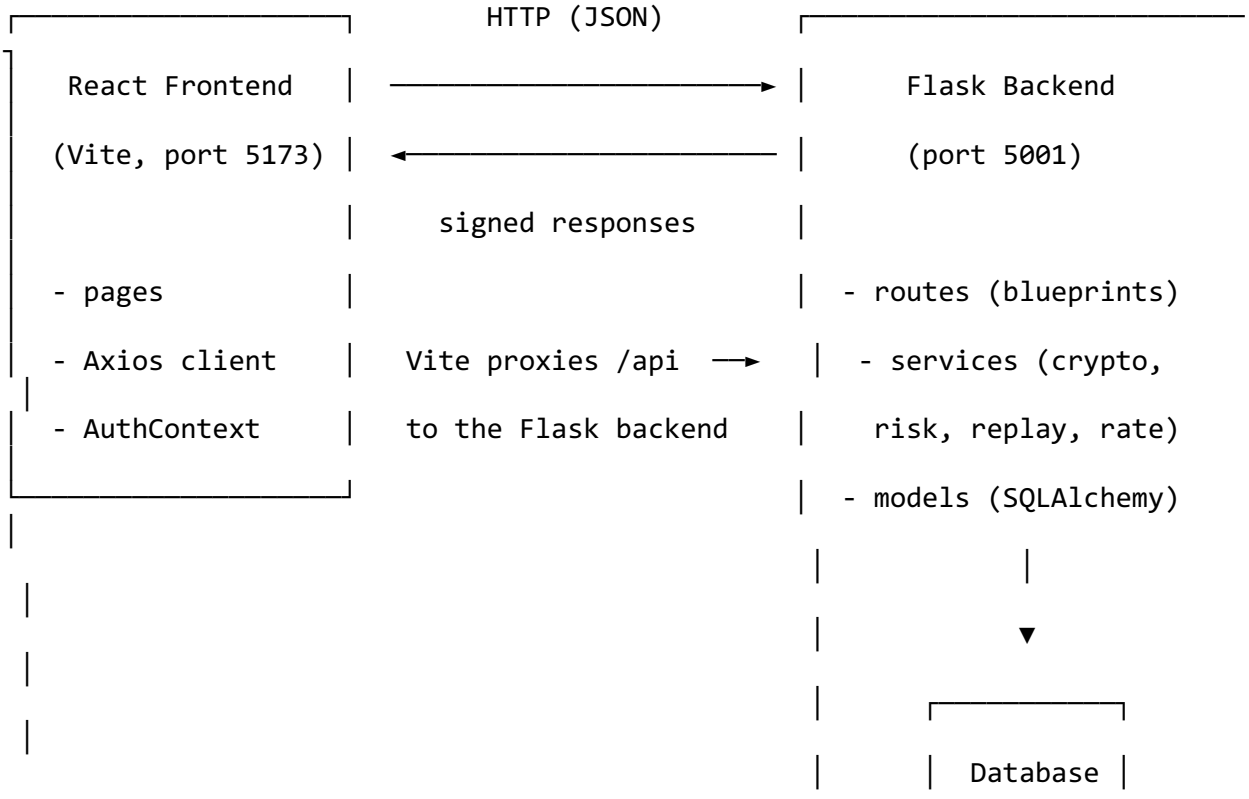
Package	Why it's here
React 19 + Vite + TypeScript	Modern, fast, type-safe UI framework and build tool.
Tailwind CSS (v4)	Utility-first styling — powers the dark glassmorphism theme.
Framer Motion	Smooth animations (page transitions, cards, the encryption animation).
Recharts	The dashboard charts (area chart, pie chart).
Lucide React	Clean, consistent icon set.
React Router	Client-side routing between pages.
Axios	HTTP client; one configured instance auto-attaches the JWT to every request.
React Hook Form	Form state + validation on Login/Register.

Database

- **SQLite** locally (zero setup, a single file) → **MySQL** in production. Because we use SQLAlchemy, the code is identical either way.

---

4. Architecture — how it all connects



Key idea: the frontend is *only* a presentation layer. All security logic lives in the Flask backend (the "security brain"). The frontend calls `/api/...`, Vite forwards that to Flask on port 5001, Flask runs the security checks, writes to the database, and returns JSON (often with a digital signature).

---

5. The security concepts (interview gold)

This is the section to actually understand. Each concept: **what it is**, **why it matters**, and **how we implemented it**.

5.1 Authentication — JWT

- **What:** A **JSON Web Token (JWT)** is a signed string the server gives you after login. You send it with every request to prove who you are. It has three parts (header.payload.signature); the payload holds your user id and role.
- **Why:** HTTP is stateless — the server doesn't "remember" you between requests. A JWT is a tamper-proof ID card you carry. If someone edits the payload, the signature no longer matches and the token is rejected.
- **How we did it:** `Flask-JWT-Extended` issues an **access token** at login (`create_access_token`) with the user's `role` and `username` embedded as claims. Protected routes use `@jwt_required()`. Tokens **expire** after a configurable time (default 30 min) so a stolen token isn't useful forever.

5.2 Password hashing — bcrypt

- **What:** We never store raw passwords. We store a **bcrypt hash**.
- **Why:** If the database leaks, attackers still can't read passwords. bcrypt is deliberately **slow** and **salted**, defeating brute-force and rainbow-table attacks.
- **How:** `User.set_password()` calls `bcrypt.hashpw()`; `check_password()` verifies with `bcrypt.checkpw()`. (See `models/user.py`.)

5.3 Authorization — Role-Based Access Control (RBAC)

- **What:** Two roles: **admin** and **client**. Admins can manage users, see all logs, rotate keys; clients see only their own data.

- **Why:** Authentication answers “who are you?”; authorization answers “what are you allowed to do?”. Both are needed.
- **How:** The role is a claim inside the JWT. The `@admin_required` decorator (`middleware/auth_guard.py`) reads that claim and blocks non-admins with a 403.

5.4 Encryption — AES-256 + RSA-2048 (hybrid encryption)

- **What:** Two kinds of encryption working together:
  - **AES-256 (symmetric):** one shared secret key encrypts *and* decrypts. Very fast — great for the actual data.
  - **RSA-2048 (asymmetric):** a public/private **key pair**. Anyone can encrypt with the public key; only the private key can decrypt. Slow, but solves the “how do both sides agree on a secret key?” problem.
- **Why hybrid:** AES is fast but both sides need the same key. RSA solves key exchange but is slow. So we **encrypt the data with AES**, then **encrypt the AES key with RSA**. This is exactly how **HTTPS/TLS** works under the hood.
- **How:** `crypto_service.full_crypto_demo()` — generates a random AES-256 key, encrypts the plaintext (AES-GCM), wraps the AES key with the RSA public key, then decrypts to prove the round-trip. Shown live on the Encryption page.

5.5 Integrity — SHA-256 hashing

- **What:** A **hash** is a fixed-length fingerprint of data. Change one character and the hash changes completely.
- **Why:** It lets us detect **tampering**. If the sender includes a hash and the gateway re-computes a different one, the payload was altered in transit.
- **How:** `crypto_service.sha256_hash()`. In the gateway, if a request’s `expected_hash` doesn’t match the computed hash → `tampered_hash` risk factor.

5.6 Digital signatures — RSA-PSS

- **What:** The gateway signs its responses with its **RSA private key**. Anyone with the **public key** can verify the signature.
- **Why:** Proves two things: **authenticity** (it really came from the gateway) and **integrity** (it wasn’t changed). A client can trust the verdict.
- **How:** `crypto_service.sign_data()` / `verify_signature()` using RSA-PSS over a SHA-256 digest. Every `/gateway/analyze` response carries a `signature`.

5.7 Replay detection — nonce + timestamp

- **What:** A **replay attack** is when an attacker captures a valid request and re-sends it (e.g., “transfer $100” replayed 50 times). A **nonce** is a “number used once.”
- **Why:** A request can be perfectly valid (good token, good hash) yet malicious because it’s a *duplicate*. Nonce + timestamp stops that.

- **How:** Each request carries a unique `nonce` and a `timestamp`. The gateway remembers seen nonces; a repeat nonce, or a timestamp outside the freshness window (default 120s), is flagged as `replay`. (See `security_state.check_replay`.)

5.8 Rate limiting — sliding window

- **What:** Cap how many requests one user/IP can make in a time window.
- **Why:** Stops flooding / abuse / brute force by volume.
- **How:** An in-memory **sliding-window counter** (`security_state.check_rate_limit`) keeps recent request timestamps per identity and rejects once they exceed the threshold (default 10 requests / 60s).

5.9 Brute-force protection

- **What:** Lock an account after too many failed logins.
- **Why:** Stops password-guessing attacks.
- **How:** `User.failed_login_attempts` increments on each failure; at the threshold (default 5) the account is auto-`is_blocked`.

5.10 Risk scoring — the "brain"

- **What:** Every request gets a **0–100 risk score** and a label: **safe / suspicious / blocked**.
- **Why:** Turns many individual checks into one clear decision, and explains *why* (which factors fired).
- **How:** `risk_engine.evaluate()` sums points per triggered factor:

Factor	Points
Invalid / Expired / Missing JWT	40
Wrong Role	30
Tampered Hash	50
Replay Attack	50
Rate Limit Exceeded	30
Blocked User	100

< 30 → safe (allow) · 30–69 → suspicious (flag) · ≥ 70 → blocked (reject)

5.11 Audit logging

- **What:** Every analyzed request and every notable event is stored.
- **Why:** Accountability, monitoring, forensics — you can answer "what happened and when?".
- **How:** `logging_service` writes to `api_logs`, `risk_scores`, `blocked_requests`, and `security_events`. The Logs page reads these.

6. The request pipeline, step by step

When a request hits `POST /api/gateway/analyze`, this runs (in `routes/gateway.py`):

```
1. JWT Verification   → valid / invalid / expired / missing
2. Role Check (RBAC)  → allowed / denied   (admin-only endpoints)
3. Replay Detection   → ok / replay        (nonce + timestamp)
4. Rate Limiting      → ok / exceeded       (sliding window)
5. SHA-256 Integrity  → match / mismatch / n-a
6. Risk Scoring       → score 0–100 + label
7. Decision           → allow / flag / reject
8. Log everything     → api_logs, risk_scores, (blocked_requests)
9. Signed Response    → RSA-PSS signature attached
```

The response JSON includes each gate's result, the risk breakdown, the final action, and the digital signature. The **Request Analyzer** and **Attack Simulator** pages both call this endpoint and visualize the result.

7. Backend — every module explained

```
backend/app/
├── __init__.py          # create_app(): the "application factory" — builds t
he
│                        #   app, loads config, connects DB + JWT + CORS,
│                        #   registers all route blueprints, creates tables,
│                        #   and seeds demo data on first run.
├── config.py            # Dev/Testing/Production settings classes. Secrets a
nd
│                        #   the database URL come from environment variables.
├── extensions.py        # The shared `db` (SQLAlchemy) and `jwt` (JWTManager)
│                        #   objects, kept here to avoid circular imports.
├── seed.py              # Creates the two roles + demo admin/client accounts.
│
├── models/              # Database tables as Python classes:
│   ├── role.py          #   roles (admin / client)
│   ├── user.py          #   users (+ bcrypt password methods)
│   ├── api_log.py       #   one row per analyzed request (the audit trail)
│   ├── security_event.py#   notable events for the dashboard timeline
│   ├── risk_score.py    #   per-request risk breakdown (factors as JSON)
│   ├── blocked_request.py#  focused record of rejected requests
│   └── refresh_token.py #   refresh-token bookkeeping
│
├── services/            # Business logic (kept out of route files):
│   ├── crypto_service.py#   AES-256, RSA-2048, SHA-256, signatures, full dem
o
│   ├── security_state.py#   in-memory runtime state: RSA keypair, mutable
```

```
│   │                       #     settings, rate-limiter + replay caches
│   ├── risk_engine.py      #   factor weights → score → label → decision
│   └── logging_service.py#  writes the audit trail across tables
│
├── middleware/
│   └── auth_guard.py     # @admin_required decorator (RBAC enforcement)
│
└── routes/               # API endpoints grouped by feature ("blueprints"):
    ├── health.py         #   /api/health liveness check
    ├── auth.py           #   register / login / me / logout / refresh
    ├── gateway.py        #   /analyze + /simulate (the core pipeline)
    ├── crypto.py         #   public key + encryption demo
    ├── dashboard.py      #   analytics: summary, timeline, risk, events, stat
us
    ├── logs.py           #   paginated/filtered logs + CSV export
    ├── admin.py          #   users, block/unblock, threats, rotate keys
    └── settings.py       #   read/update runtime security settings
```

Why the "application factory" pattern? Instead of one giant file, `create_app()` assembles the app inside a function. This makes it testable, avoids circular imports, and cleanly separates dev/test/prod configuration — how professional Flask apps are structured.

---

8. Backend — every API endpoint

Base URL locally: `http://localhost:5001`

Method	Endpoint	Auth	What it does
GET	`/api/health`	—	Liveness check (is the server up?)
POST	`/api/auth/register`	—	Create account (bcrypt hash), returns JWT
POST	`/api/auth/login`	—	Verify credentials, returns JWT; brute-force lockout
GET	`/api/auth/me`	JWT	Current user's profile
POST	`/api/auth/logout`	JWT	Records a logout event
POST	`/api/auth/refresh`	refresh JWT	Issues a new access token
POST	`/api/gateway/analyze`	—	Run a crafted

Method	Endpoint	Auth	What it does
			request through all gates
POST	`/api/gateway/simulate`	—	Server-crafted attack scenarios (Attack Simulator)
GET	`/api/crypto/public-key`	—	The gateway's RSA public key
POST	`/api/crypto/demo`	JWT	Full AES+RSA+SHA-256+signature demo
GET	`/api/dashboard/summary`	JWT	Counts: total/allowed/blocked/etc.
GET	`/api/dashboard/timeline`	JWT	Requests grouped by hour (activity chart)
GET	`/api/dashboard/risk-distribution`	JWT	Safe/suspicious/blocked breakdown (pie)
GET	`/api/dashboard/recent-events`	JWT	Latest security events (timeline feed)
GET	`/api/dashboard/top-endpoints`	JWT	Most-hit endpoints
GET	`/api/dashboard/security-status`	JWT	JWT/encryption/logging/replay status
GET	`/api/logs`	JWT	Paginated, searchable, filterable audit log
GET	`/api/logs/export`	JWT	Download logs as CSV
GET	`/api/admin/users`	admin	List all users
POST	`/api/admin/users/<id>/block`	admin	Block a user
POST	`/api/admin/users/<id>/unblock`	admin	Unblock a user
GET	`/api/admin/threats`	admin	Threat overview counts + recent blocked
POST	`/api/admin/rotate-keys`	admin	Regenerate the RSA key pair

Method	Endpoint	Auth	What it does
GET	`/api/settings`	—	Read runtime security settings
PUT	`/api/settings`	admin	Update settings (JWT expiry, rate limit, etc.)

---

9. Frontend — every page explained

The app has a **public** area and a **protected** dashboard (`/app/...`). After login the JWT is stored in `localStorage` and auto-attached to every API call.

Page	Route	What it shows / does
Landing	`/`	Marketing hero: pitch, feature cards, “Open Dashboard” CTA.
Login	`/login`	Split-screen sign-in, validation, show-password, demo-account hint.
Register	`/register`	Create account with role select and validation.
Dashboard	`/app/dashboard`	The control center: 6 summary cards, 24h activity area-chart, risk-distribution pie, recent security events, security-status panel, quick actions.
Request Analyzer	`/app/analyzer`	Craft a request (endpoint, payload, JWT, nonce, timestamp) → see each gate’s result, risk score, and the signed decision. “Auto-fill valid” fills a real token.
Encryption	`/app/encryption`	Type plaintext → watch AES encrypt, RSA wrap the key, SHA-256 hash, digital signature, and the decrypted round-trip. Animated lock.
Attack Simulator	`/app/attack-simulator`	One click launches Invalid JWT / Expired JWT / Replay / Tampered Payload / Wrong

Page	Route	What it shows / does
		Role / Rate-Limit attacks and shows how the gateway blocks/flags each.
Risk Center	`/app/risk`	A gauge of average risk, safe/suspicious/blocked counts, the scoring model, and recent scored requests.
Logs	`/app/logs`	Searchable, filterable, paginated audit table with colored status badges and **CSV export**.
Admin Console	`/app/admin`	(Admin only) Threat overview cards, user management (block/unblock), key rotation, recent blocked list.
Settings	`/app/settings`	Runtime security controls: JWT expiry, rate limit, replay window, toggles. Admin can save.

Shared UI: a fixed **sidebar** (nav) + **topbar** (user, role badge, logout), reusable glassmorphism `Card`, `Badge`, `StatCard`, `Skeleton`, `Spinner` components, and a global `AuthContext` that tracks the logged-in user.

---

10. Database tables

Table	Holds
`users`	id, username, email, bcrypt password hash, role, is_blocked, failed_login_attempts, created_at
`roles`	id, name (`admin`/`client`), description
`api_logs`	per-request audit row: endpoint, method, per-gate statuses, overall status, risk score/label, action, reason, ip, timestamp
`security_events`	timeline feed: event_type, severity, description, user, timestamp
`risk_scores`	per-request score, label, and the contributing factors (JSON)
`blocked_requests`	rejected requests: endpoint, reason, risk

Table	Holds
	score, ip
`refresh_tokens`	issued refresh-token ids (for revocation bookkeeping)

On first run the app **auto-creates all tables** and **seeds** two roles and two demo accounts — no manual SQL needed.

---

11. How to run it from scratch

Prerequisites

- **Python 3.11+** (built on 3.13)
- **Node.js 18+** (built on Node 22) and npm
- No database to install — local dev uses SQLite (built into Python).

You need **two terminals**: one for the backend, one for the frontend.

Terminal 1 — Backend (Flask API)

```
cd secure_api_gateway_v2/backend

# 1. Create an isolated Python environment
python3 -m venv venv

# 2. Activate it
source venv/bin/activate            # macOS/Linux
# venv\Scripts\activate             # Windows (PowerShell)

# 3. Install dependencies
pip install -r requirements.txt

# 4. Create your local config from the template
cp .env.example .env

# 5. Start the server
python run.py
```

The backend runs at **http://localhost:5001**. Verify:

```
curl http://localhost:5001/api/health
# {"status":"ok","service":"secure-api-gateway",...}
```

On first start it prints the seeded demo accounts.

Terminal 2 — Frontend (React dashboard)

```
cd secure_api_gateway_v2/frontend

# 1. Install dependencies
npm install

# 2. Start the dev server
npm run dev
```

Open **http://localhost:5173** in your browser.

> The frontend proxies all `/api` calls to the backend on port 5001, so the backend **must** be running too.

Demo accounts (created automatically)

Role	Username	Password
Admin	`admin`	`admin123`
Client	`client`	`client123`

> ⚠️ Port note: we use **5001**, not 5000 — on macOS, port 5000 is taken by AirPlay Receiver.

---

12. How to demo it (live walkthrough script)

A 3–4 minute demo that shows everything working:

1. **Landing page** → click **Open Dashboard**.
2. **Login** as `admin` / `admin123`. (Point out: JWT is issued and stored.)
3. **Dashboard** → "This is the security posture at a glance — total requests, blocked, suspicious, failed logins, risk distribution, live events."
4. **Attack Simulator** → click **Replay Attack** → "The gateway detected a reused nonce and flagged it — risk score 50." Try **Tampered Payload** and **Invalid JWT** too.
5. **Request Analyzer** → click **Auto-fill valid** → **Analyze** → "Clean request: all gates pass, risk 0, allowed, and the response is digitally signed." Then delete the token and re-run → "Now it's flagged."
6. **Encryption** → type a message → **Encrypt & Sign** → "AES encrypts the data, RSA wraps the key, SHA-256 hashes it, and it's signed — then decrypted back."
7. **Logs** → "Every request we just made is here — searchable, filterable, exportable to CSV."
8. **Admin Console** → "Admins can block users and rotate the RSA keys."
9. Refresh the **Dashboard** → "The charts and counts updated with everything we just did."

---

13. Interview Q&A

Q: What is a JWT and why use it? A stateless, signed token issued at login carrying the user's id and role. The server doesn't store sessions; it just verifies the signature. If the payload is tampered with, verification fails. We set short expiries so a leaked token isn't useful for long.

Q: Why bcrypt instead of SHA-256 for passwords? SHA-256 is *fast*, which is bad for passwords (attackers can try billions/sec). bcrypt is deliberately slow and salted, so brute force and rainbow tables become impractical. (We use SHA-256 for *data integrity*, a different job.)

Q: Explain your hybrid encryption. AES-256 encrypts the actual data (fast). The random AES key is then encrypted with the recipient's RSA-2048 public key (secure key exchange). Only the private key can unwrap it. This is how TLS works — symmetric speed + asymmetric key exchange.

Q: How do you stop replay attacks without Redis? Each request carries a unique nonce and a timestamp. The gateway remembers seen nonces and rejects duplicates, and rejects requests whose timestamp is outside a freshness window. Simple and explainable.

Q: How does rate limiting work here? A per-user/IP sliding-window counter in memory: we keep the timestamps of recent requests and reject once they exceed the threshold within the window.

Q: What's the risk score for? It aggregates all the individual gate results into one 0–100 number and a label (safe/suspicious/blocked) that drives the allow/flag/reject decision — and it records *why* (which factors fired), which is great for auditing.

Q: What does the digital signature prove? Authenticity (the response really came from the gateway) and integrity (it wasn't altered). Clients verify it with the gateway's public key.

Q: Why a gateway instead of putting this in the backend? Centralization. One consistent checkpoint enforces auth, integrity, and rate limits for every endpoint, instead of each service reinventing security.

Q: How is the code structured / is it maintainable? Application-factory pattern, feature-based blueprints, business logic in a `services/` layer, models via an ORM, RBAC in reusable middleware. The frontend mirrors this with reusable components, a typed API layer, and an auth context.

---

14. Troubleshooting

Problem	Fix
`Port 5000 is in use` / AirPlay	We already use **5001**. If 5001 is busy, set

Problem	Fix
	`PORT=5002` in `backend/.env`.
Frontend loads but API calls fail	Make sure the **backend is running** on 5001 (the frontend proxies `/api` to it).
`ModuleNotFoundError` in backend	Activate the venv (`source venv/bin/activate`) and re-run `pip install -r requirements.txt`.
`npm run dev` errors	Run `npm install` first; ensure Node 18+.
Want a clean slate	Stop the backend and delete `backend/instance/gateway_dev.db`; it re-seeds on next start.
Login says "invalid credentials"	Use the seeded accounts (`admin`/`admin123`), or register a new one.

---

15. Project structure

```
secure_api_gateway_v2/
├── README.md                 # ← this file
├── docs/
│   └── BLUEPRINT.md          # the refined project blueprint (v3 plan)
├── backend/
│   ├── app/                  # the Flask application (see §7)
│   ├── run.py                # entry point (starts the server)
│   ├── requirements.txt      # Python dependencies
│   ├── .env.example          # config template
│   └── README.md             # backend quick reference
└── frontend/
    ├── src/
    │   ├── pages/            # the 11 pages (see §9)
    │   ├── components/       # sidebar, topbar, layout, reusable UI
    │   ├── context/          # AuthContext (who is logged in)
    │   ├── services/         # Axios API client
    │   └── types/            # shared TypeScript types
    ├── vite.config.ts        # dev server + /api proxy to backend
    └── package.json
```

---

16. Roadmap / planned refinements

The current code is the **v2** build described above. A refined **v3 blueprint** (see `docs/BLUEPRINT.md`) tightens scope for a B.Tech project and adds a few features. Planned changes (not yet applied to the code):

- **Remove:** refresh tokens, key rotation (keep RSA keys static per run).

- **Change:** move replay detection from in-memory to a **MySQL nonces table**.
- **Trim:** Admin Console (users, logs, export, block/unblock only) and Settings (JWT expiry, rate-limit threshold, theme toggle only).
- **Add:** API Playground, Request Flow Visualizer, API Documentation page, Security Health Score, and Endpoint Management (enable/disable endpoints).
- **Deploy:** host online (no Docker) — managed MySQL + Flask on a PaaS + static React on Vercel/Netlify.

Built as a final-year B.Tech Information Technology project. Modern UI, real security concepts, one cohesive product.